

```

def cheapest_walk(self, start, end):
    """
    Finds the lowest cost path from start to end using Bellman-Ford algorithm
    :param start: the starting vertex
    :param end: the ending vertex
    :return: (list of vertices in the path, total cost), or (None, None) if unreachable
    raises an exception if there are negative cost cycles accessible from the starting
    vertex or if the vertices do not exist
    """
    if not self.valid_vertex(start):
        raise ValueError("Vertex {} does not exist.".format(start))
    if not self.valid_vertex(end):
        raise ValueError("Vertex {} does not exist.".format(end))
    dist={} # dist[x]=the cost of the minimum cost walk from start to x at one point
    prev={} # prev[x]=the vertex just before x in the walk
    vertices=self.parse_vertices()
    for node in vertices:
        dist[node] = float("inf") #initialize dist[node] with infinity in order to
compare and find the cheapest walk
    dist[start] = 0 #the cost of the walk from start to itself is 0
    #relaxing all edges max no of vertices-1
    #because the shortest possible path between any two nodes can only have no of
vertices-1 edges
    changed=True
    max_iter=len(vertices)-1
    index=0
    while changed and index < max_iter :
        changed = False
        for (x,y) in self._costs.keys():
            if dist[x]!=float("inf"):
                if dist[x]+self.get_cost(x,y) < dist[y]:
                    dist[y] = dist[x]+self.get_cost(x,y)
                    prev[y]=x
                    changed = True
        index+=1
    #check for negative cost cycles
    #if one more relax can be done it means we have a negative cycle
    for (x, y) in self._costs.keys():
        if dist[x] != float("inf"):
            if dist[x] + self.get_cost(x, y) < dist[y]:
                raise ValueError("Negative weight cycle detected from vertex", start)
    #check if the end is reachable
    if dist[end] == float('inf'):
        return (None, None)
    else:
        #retrive walk
        walk = []
        current = end
        while current is not None:
            walk.append(current)
            if current == start:
                break
            current = prev[current] # move one step back
        walk.reverse() # reverse the list to show path from start to end
        return walk, dist[end]

```